

DD: PLM-CFG-02 — Tax Configuration

Developer implementation guide: DD_PLM-CFG-02-Tax_Configuration-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_PLM-CFG-02-Tax_Configuration-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group I:** Catalog/Configuration | **Version:** v1.0 Configuration DD owning all tax rate definitions and cascade rules used at charge time. Every charge in BIL-01 Core calls PLM-CFG-02's tax computation service (per R-BIL-01-AM-5) to get a tax line breakdown given an operator, a taxable base, and the charge context (subscription, customer category, effective date). The service returns one tax line per applicable tax rate, with the rate, the base it was computed against, and the resulting amount. BIL-01 attaches these tax lines to the charge attempt and forwards them to BIL-02-GEN-01 (POSTPAID) or accounts them as part of the wallet debit (PREPAID). **Cleanly separated from BIL-02-TAX-01:** TAX-01 owns tax INVOICE generation and regulator signing/registration AFTER payment (eTIMS, EFD, etc.). PLM-CFG-02 owns tax COMPUTATION at charge time. The two never overlap; they cover sequential phases of the same lifecycle. **Version history:** see CHANGELOG_PLM-CFG-02-Tax_Configuration.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_PLM-CFG-02-Tax_Configuration-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/tax/admin/groups/{code}/members/{rule_code}
- GET /api/tax/admin/groups
- GET /api/tax/admin/rules?operatorCode=WIK&activeOn=2026-05-18
- POST /api/tax/admin/applicability-rules

- POST /api/tax/admin/groups
- POST /api/tax/admin/groups/{code}/members
- POST /api/tax/admin/rules
- POST /api/tax/compute
- PUT /api/tax/admin/groups/{code}

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- tax-applicability
- tax-configuration

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group I:** Catalog/Configuration | **Version:** v1.0

Configuration DD owning all tax rate definitions and cascade rules used at charge time. Every charge in BIL-01 Core calls PLM-CFG-02's tax computation service (per R-BIL-01-AM-5) to get a tax line breakdown given an operator, a taxable base, and the charge context (subscription, customer category, effective date). The service returns one tax line per applicable tax rate, with the rate, the base it was computed against, and the resulting amount. BIL-01 attaches these tax lines to the charge attempt and forwards them to BIL-02-GEN-01 (POSTPAID) or accounts them as part of the wallet debit (PREPAID).

Cleanly separated from BIL-02-TAX-01: TAX-01 owns tax INVOICE generation and regulator signing/registration AFTER payment (eTIMS, EFD, etc.). PLM-CFG-02 owns tax COMPUTATION at charge time. The two never overlap; they cover sequential phases of the same lifecycle.

Version history: see `CHANGELOG_PLM-CFG-02-Tax_Configuration.md` in the same directory.

Glossary

Term	Meaning
Tax rule	A row in <code>tax_rule</code> defining: operator, code (e.g. <code>WIK_INTERNET_EXCISE</code>), rate (e.g. 0.15), the base method, the effective date range, the regulator reference.
Tax group	A logical grouping of tax rules that apply together to a taxable thing. Example: <code>WIK_INTERNET</code> groups <code>WIK_INTERNET_EXCISE</code> (order 1) and <code>WIK_INTERNET_VAT</code> (order 2). A taxable charge resolves to exactly one tax group via applicability rules.
Cascade	The compound application of multiple taxes where later taxes are computed on a base that includes earlier taxes. In Kenya, VAT on Internet/Voice cascades over excise: <code>tax = excise + VAT(base + excise)</code> .
Base method	The enum determining what amount a tax rate is applied against. Values: <code>BASE</code> (charge base only), <code>BASE_PLUS_PRIOR</code> (base + sum of all prior taxes in the same group), <code>BASE_PLUS_EXCISE</code> (base + excise-tagged prior tax only — used for jurisdictions where some prior taxes are excluded from the VAT base).
Taxable	Anything that can incur tax: a service charge, a package fee, a fee for service, a one-shot purchase. Identified by <code>taxable_kind</code> (<code>SERVICE / PACKAGE / FEE / ONESHOT</code>) and a <code>taxable_ref</code> (e.g., a Service code from PLM-CFG-01, a Package code from SIP-01, a BillableEvent code from BIL-CFG-01).
Applicability rule	A Drools rule (DRL) that decides whether a given tax group applies to a given charge context. Evaluated by the tax compute service.
Tax line	The output of one tax rule application: <code>{ code, rate, base_amount, tax_amount }</code> . A charge produces zero or more tax lines depending on the resolved tax group.
Regulator	The authority that owns a particular tax code. Examples: KRA (Kenya Revenue Authority), URA (Uganda Revenue Authority), TRA (Tanzania Revenue Authority), DGID (Direction Générale des Impôts et Domaines — Senegal).


```
base=BASE_PLUS_PRIOR,  
order=2  
  
3. Apply in order:  
- Excise: 10000 × 0.15  
    = 1500 (base 10000)  
- VAT: (10000+1500) × 0.16  
    = 1840 (base 11500)  
  
4. Total tax: 3340  
   Total with tax: 13340
```

▼

```
{ taxLines: [  
  { code: WIK_INTERNET_EXCISE, rate: 0.15,  
    baseAmount: 10000.00, taxAmount: 1500.00 },  
  { code: WIK_INTERNET_VAT, rate: 0.16,  
    baseAmount: 11500.00, taxAmount: 1840.00 } ],  
totalTaxAmount: 3340.00,  
totalWithTax: 13340.00,  
currency: KES }
```

▼

BIL-01 records tax in charge_ledger.tax_amount_total
and BIL-02-GEN-01 stores tax lines per invoice line

2. Business rules

Rules grouped: CT (catalog), AP (applicability), CO (computation), CA (cascade), R0 (rounding), DA (effective date), EX (exemptions and overrides), EV (event emission — config invalidation).

Rule group CT — Catalog

ID	Rule	Triggered on
R-PLM-02-CT-1	Each tax rule is a row in <code>tax_rule</code> with a globally unique code (e.g., <code>WIK_INTERNET_EXCISE</code>). The code MUST identify the operator (3-letter prefix), the taxable category, and the tax type. Rules CANNOT be deleted; they are deactivated by setting <code>effective_until</code> to a past date. Historical charges may reference deactivated rules — the rule definition is retained for audit.	Every config write
R-PLM-02-CT-2	Each tax group is a row in <code>tax_group</code> with a unique code (e.g., <code>WIK_INTERNET</code>). The group lists which <code>tax_rule</code> codes belong to it, in <code>order_within_group</code> . The same <code>tax_rule</code> MAY appear in multiple groups (uncommon but supported — e.g., a generic VAT rule used across multiple categories).	Every config write
R-PLM-02-CT-3	Each operator has its own set of tax rules and tax groups. Cross-operator references (e.g., a Wananchi KE rule used by Wananchi UG) are forbidden — even if rates happen to match. Each operator's rules carry their own code and <code>regulator_reference</code> .	Every config write
R-PLM-02-CT-4	A tax rule MUST declare a <code>regulator</code> (KRA / URA / TRA / DGID / etc.) and a <code>regulator_tax_code</code> (the code the regulator uses to identify the tax in their own systems — used by BIL-02-TAX-01 when generating tax invoices for eTIMS/EFD/etc.). This field is informational at compute time but mandatory for downstream regulator-facing flows.	Every rule write

Rule group AP — Applicability

ID	Rule	Triggered on
R-PLM-02-AP-1	Applicability resolution determines, for a given charge context, which tax_group applies. Resolution is via Drools (tax-applicability KJAR). Each applicability rule declares: a tax_group code, a priority (1=highest), and a DRL condition (matching on operator, taxable_kind, taxable_ref patterns, customer_category, customer_attribute map, and taxable_at). The first matching rule (highest priority) wins.	Every compute call

ID	Rule	Triggered on
R-PLM-02-AP-2	If no applicability rule matches, the compute service returns <code>taxLines: []</code> with <code>totalTaxAmount: 0</code> and a <code>resolutionStatus: NO_TAX_GROUP_RESOLVED</code> flag. This is a legitimate outcome (e.g., a non-taxable internal fee) — BIL-01 does not treat it as an error. An audit log is written so finance can review zero-tax charges.	No applicability match
R-PLM-02-AP-3	An applicability rule MAY declare <code>tax_group = NONE</code> explicitly — this is how operators model tax-exempt categories (e.g., government customers exempt from VAT in some jurisdictions). Resolution status returned: <code>EXEMPT_BY_RULE</code> with the rule reference.	Explicit exemption rule
R-PLM-02-AP-4	Multiple applicability rules MAY match — the priority field disambiguates. Ties on priority are forbidden at write time (rejected with <code>DUPLICATE_PRIORITY_IN_TAX_RULESET</code>).	Config write
R-PLM-02-AP-5	Applicability rules are scoped by operator: when evaluating for <code>operator_code = WIK</code> , only rules with <code>operator_code = WIK</code> are loaded. There is no global rule.	Every compute

Rule group CO — Computation

ID	Rule	Triggered on
R-PLM-02-CO-1	Once a tax group is resolved, the compute service iterates the group's tax rules in <code>order_within_group</code> ascending. For each rule: (a) compute the base per the rule's <code>base_method</code> ; (b) apply the rate (<code>base × rate</code>); (c) round per the group's <code>rounding_mode</code> and <code>rounding_scale</code> ; (d) record the tax line <code>{code, rate, baseAmount, taxAmount}</code> ; (e) carry forward the running total of taxes for use by subsequent <code>BASE_PLUS_PRIOR</code> rules.	Every compute
R-PLM-02-CO-2	The compute service is stateless — it does not write to its own DB during a compute call. It reads <code>tax_rule</code> , <code>tax_group</code> , and the applicability KJAR. Caching is allowed (R-CT-5 below). All persistence happens in BIL-01's <code>charge_ledger</code> and BIL-02's invoice line tables.	Every compute
R-PLM-02-CO-3	Compute service requires <code>operator_code</code> , <code>taxable_kind</code> , <code>taxable_ref</code> , <code>base_amount</code> , <code>currency</code> , <code>customer_category</code> , <code>taxable_at</code> in the request. Missing any required field → 400 <code>MISSING_REQUIRED_FIELD</code> .	Request validation
R-PLM-02-CO-4	<code>base_amount</code> MUST be non-negative. A negative input would imply a refund/credit-note path; tax on credit notes is computed by BIL-02-ADJ-01 (which calls PLM-CFG-02 with the absolute value and the resulting tax becomes a tax credit in the credit note). Negative inputs are rejected with <code>NEGATIVE_BASE_NOT_SUPPORTED</code> .	Request validation
R-PLM-02-CO-5	Compute service is idempotent at the request level: the same input always produces the same output for a given <code>taxable_at</code> date. No request-level idempotency key is needed because there are no side effects. Callers (BIL-01 / BIL-02-ADJ-01) handle their own idempotency.	Behavior

Rule group CA — Cascade base methods

ID	Rule	Triggered on
R-PLM-02-CA-1	<code>base_method = BASE</code> : the tax is computed on the charge base amount only. Example: TV in Kenya — only VAT applies, on the package base. Tax amount = <code>base × rate</code> .	<code>base_method = BASE</code>

ID	Rule	Triggered on
R-PLM-02-CA-2	base_method = BASE_PLUS_PRIOR: the tax is computed on (base + sum of all tax_amount of prior rules in the same group). Used in cascading systems (Kenya Internet, Kenya Voice). Tax amount = (base + Σ prior_tax) $\times$ rate.	base_method = BASE_PLUS_PRIOR
R-PLM-02-CA-3	base_method = BASE_PLUS_EXCISE: the tax is computed on (base + sum of tax_amount of prior rules where rule.tax_type = EXCISE). Used in jurisdictions where VAT cascades over excise specifically but NOT over a separate prior surcharge. Tax amount = (base + Σ excise_tax) $\times$ rate.	base_method = BASE_PLUS_EXCISE
R-PLM-02-CA-4	A tax rule MUST declare a tax_type enum: EXCISE / VAT / SURCHARGE / LEVY / WITHHOLDING_INFO. This is informational for cascading (used by BASE_PLUS_EXCISE) and for regulator reporting. Most jurisdictions use EXCISE and VAT; surcharge and levy cover regulator-specific taxes (e.g., universal service fund levy).	Every rule write

Rule group RO — Rounding

ID	Rule	Triggered on
R-PLM-02-RO-1	Each tax group declares rounding_mode (Java RoundingMode enum: HALF_EVEN default, HALF_UP, HALF_DOWN, etc.) and rounding_scale (number of decimal places — typically 2 for currencies with sub-units). Rounding is applied to each tax line's tax_amount individually before summation.	Every tax line
R-PLM-02-RO-2	The totalTaxAmount in the response is the sum of rounded tax line amounts, NOT a separate rounding of the unrounded sum. This means $\text{totalTaxAmount} = \Sigma \text{round}(\text{tax_line_i})$, which can differ by 0.01 from $\text{round}(\Sigma \text{unrounded_tax_line_i})$. Documented behavior — finance reconciliation expects this.	Response computation
R-PLM-02-RO-3	For zero-decimal currencies (UGX has no sub-units), rounding_scale MUST be 0. For two-decimal currencies (KES, TZS, XOF), rounding_scale is typically 2. Config validation enforces this at write time per the operator's currency.scale (resolved from the operator deployment metadata).	Config write

Rule group DA — Effective date

ID	Rule	Triggered on
R-PLM-02-DA-1	Every tax rule has effective_from (mandatory, TIMESTAMP) and effective_until (nullable; NULL = open-ended). The compute service uses the request's taxable_at timestamp to pick the rule version active at that moment.	Every compute
R-PLM-02-DA-2	Multiple versions of the same logical rule (e.g., WIK_INTERNET_EXCISE rate changed from 10% $\rightarrow$ 15% on 2023-07-01) coexist in tax_rule as separate rows with overlapping codes BUT non-overlapping effective windows. Code+effective windows must be non-overlapping; enforced via a DB exclusion constraint.	Config write
R-PLM-02-DA-3	If no rule version is active on taxable_at (gap in effective dates), the resolution status is RULE_GAP_AT_DATE and the tax group's other rules still apply. A configuration warning is logged for the operator's config admin. This is a defensive case — careful config management avoids it.	Compute
R-PLM-02-DA-4	Tax group memberships (tax_group_member rows) are NOT time-bound at the group level — the group itself is a stable container. Time variability lives at the individual tax_rule row level.	Behavior

Rule group EX — Exemptions and overrides

ID	Rule	Triggered on
R-PLM-02-EX-1	Customer-level tax exemption is signaled via <code>customer_attribute.tax_exempt = true</code> in the compute request. When this attribute is set, the compute service skips all tax rule application and returns <code>taxLines: []</code> with <code>resolutionStatus: EXEMPT_BY_CUSTOMER_ATTRIBUTE</code> . The audit log captures the customer attribute presence.	Tax-exempt customer
R-PLM-02-EX-2	An admin override at compute time (e.g., for a special one-off charge that should not bear tax) is signaled via <code>forceTaxFree: true</code> in the request. Requires the caller to have <code>TAX_OVERRIDE</code> role (the caller checks this; the compute service trusts the flag). Returns <code>taxLines: []</code> with <code>resolutionStatus: ADMIN_FORCE_TAX_FREE</code> and the actor recorded.	Admin override
R-PLM-02-EX-3	Exemptions and overrides do NOT change the tax catalog state — they affect only the single compute call. Historical compute outputs are reproducible because the catalog state is preserved.	Behavior

Rule group EV — Event emission (config changes)

ID	Rule	Triggered on
R-PLM-02-EV-1	Changes to <code>tax_rule</code> , <code>tax_group</code> , <code>tax_group_member</code> , or <code>tax_applicability_rule</code> emit a Kafka event on <code>sophix.config.tax.changed</code> with payload <code>{operator_code, change_type (RULE_ADDED / RULE_DEACTIVATED / GROUP_CHANGED / APPLICABILITY_RULE_CHANGED), affected_codes: [...]}</code> . Consumers (BIL-01 Core, BIL-02-GEN-01, ADJ-01) use these events to invalidate their PLM-CFG-02 read caches.	Every config write
R-PLM-02-EV-2	The compute service maintains a local cache of tax rules and applicability rulesets per operator (R-CO-2). Default cache TTL is 300 seconds. The cache MUST be invalidated on receipt of <code>sophix.config.tax.changed</code> . Operators changing tax rates need to see the change applied within the cache TTL window at worst.	Cache invalidation

3. Module owner and impacted modules

Role	Module	What it does
Owner	New tax-configuration module (Java/Spring, deployed as part of the catalog cluster)	Owns <code>tax_rule</code> , <code>tax_group</code> , <code>tax_group_member</code> , <code>tax_applicability_rule</code> tables. Hosts the Drools tax-applicability KJAR. Exposes <code>POST /api/tax/compute</code> (synchronous compute) and <code>/api/tax/admin/*</code> (admin CRUD on tax catalog).
Impacted (callees)	Drools KIE Server (DRL evaluation), operator config DB	Receive evaluation requests.
Impacted (callers)	BIL-01 Core (per R-BIL-01-AM-5 — every charge that produces a non-zero gross), BIL-02-ADJ-01 (when issuing tax-bearing credit/debit notes — calls compute with absolute base value), BIL-02-GEN-01 (for invoice-level tax recomputation in edge cases), backoffice admin UI (CRUD on catalog)	Issue compute or admin requests.
Impacted (downstream consumers)	All callers consume the compute response inline. The <code>sophix.config.tax.changed</code> event is consumed by BIL-01 Core's catalog cache, BIL-02-GEN-01, and BIL-02-ADJ-01 for cache invalidation.	—
Engines	Drools (DRL applicability evaluation), Kafka (config change events)	—

4. Foundation references

DB → FOUNDATION_DB.md · Drools → FOUNDATION_DROOLS.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md (Redis for catalog cache) · Auth → FOUNDATION_AUTH.md (introduces TAX_CATALOG_ADMIN, TAX_OVERRIDE roles).

Technical design

Code structure

```
modules/tax-configuration/
  src/main/java/com/sophix/catalog/tax/
    service/
      TaxComputeService.java      ← stateless compute orchestrator
      ApplicabilityResolver.java  ← Drools wrapper, resolves tax_group for a context
      CascadeEngine.java          ← iterates rules, applies base methods, rounds
      TaxRuleCache.java           ← Redis-backed cache with invalidation hook
    rest/
      TaxComputeResource.java     ← POST /api/tax/compute
      TaxAdminResource.java       ← admin CRUD on rule/group/applicability
    domain/
      TaxRule.java
      TaxGroup.java
      TaxApplicabilityRule.java
      TaxLine.java
      BaseMethod.java             ← enum: BASE, BASE_PLUS_PRIOR, BASE_PLUS_EXCISE
      TaxType.java                ← enum: EXCISE, VAT, SURCHARGE, LEVY, WITHHOLDING_INFO
      ResolutionStatus.java        ← enum: RESOLVED, NO_TAX_GROUP_RESOLVED, EXEMPT_BY_RULE,
                                   EXEMPT_BY_CUSTOMER_ATTRIBUTE, ADMIN_FORCE_TAX_FREE, RULE_GAP_AT_DATE
    repository/
      TaxRuleRepository.java
      TaxGroupRepository.java
      TaxApplicabilityRuleRepository.java
    kafka/
      TaxConfigChangedPublisher.java
  src/main/resources/
    config/liquibase/changelog/tax-configuration/
      010_tax_rule.xml
      011_tax_group.xml
      012_tax_group_member.xml
      013_tax_applicability_rule.xml
    drl/tax-applicability/
      WIK_applicability.drl
      WUG_applicability.drl
      WTZ_applicability.drl
      YASN_applicability.drl
```

Data model

tax_rule

Column	Type	Notes
code	TEXT NOT NULL	Globally unique code prefixed with operator. Example: WIK_INTERNET_EXCISE, WIK_INTERNET_VAT, WIK_TV_VAT, WIK_VOICE_EXCISE, WIK_VOICE_VAT, WUG_GENERAL_VAT, YASN_FCS_LEVY. The 3-letter operator prefix matches operator_code
effective_from	TIMESTAMP NOT NULL	Inclusive lower bound. Example: 2023-07-01 00:00:00 (rate hike effective date)
effective_until	TIMESTAMP NULL	Exclusive upper bound. NULL = open-ended (currently active). Example: 2023-07-01 00:00:00 (the old rule version is closed at exactly the moment the new one opens)
operator_code	TEXT NOT NULL	Enum. Values: WIK (Wananchi KE), WUG (Wananchi UG), WTZ (Wananchi TZ), YASN (Yas Senegal)

Column	Type	Notes
tax_type	TEXT NOT NULL	Enum used by cascade base methods and regulator reporting. Values: EXCISE (excise tax — Kenya internet/voice) / VAT (Value Added Tax / Sales Tax) / SURCHARGE (regulator-specific surcharge) / LEVY (e.g., universal service fund levy) / WITHHOLDING_INFO (informational tax line that does NOT add to total, used to display withholding obligations to corporate customers)
rate	NUMERIC(8,6) NOT NULL	Tax rate as a decimal fraction (0.15 = 15%). Example: 0.150000 for excise, 0.160000 for VAT, 0.005000 for a 0.5% levy
regulator	TEXT NOT NULL	Enum. Values: KRA (Kenya Revenue Authority) / URA (Uganda Revenue Authority) / TRA (Tanzania Revenue Authority) / DGID (Direction Générale des Impôts et Domaines — Senegal) / OTHER
regulator_tax_code	TEXT NOT NULL	The code the regulator uses in their own systems. Used by BIL-02-TAX-01 for eTIMS/EFD submission. Example: EXC_INT_15 for KRA's Internet excise; VAT_STD_16 for KRA's standard VAT
display_name	TEXT NOT NULL	Customer-facing name for invoice display. Example: Excise Duty (Internet), VAT 16%
description	TEXT NULL	Internal notes. Example: Per Finance Act 2023, Sec 17 — excise rate raised from 10% to 15% on internet services effective 2023-07-01
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	Row creation time. Example: 2026-05-18 10:30:00
created_by	TEXT NOT NULL	Admin user ID. Example: admin_user_42

Primary key: (code, effective_from) — composite, supporting multiple versions of the same logical rule.

Indexes: PK; (operator_code, effective_from DESC) for operator-scoped lookups; (operator_code, tax_type). Exclusion constraint: no overlapping [effective_from, effective_until) windows for the same code.

Sample data:

code	effective_from	effective_until	operator	tax_type	rate	regulator	reg_code	display_name
WIK_INTERNET_EXCISE	2023-07-01	NULL	WIK	EXCISE	0.150000	KRA	EXC_INT_15	Excise Duty (Internet)
WIK_INTERNET_EXCISE	2020-01-01	2023-07-01	WIK	EXCISE	0.100000	KRA	EXC_INT_10	Excise Duty (Internet)
WIK_INTERNET_VAT	2020-01-01	NULL	WIK	VAT	0.160000	KRA	VAT_STD_16	VAT 16%
WIK_TV_VAT	2020-01-01	NULL	WIK	VAT	0.160000	KRA	VAT_STD_16	VAT 16%
WIK_VOICE_EXCISE	2020-01-01	NULL	WIK	EXCISE	0.100000	KRA	EXC_VOICE_10	Excise Duty (Voice)
WIK_VOICE_VAT	2020-01-01	NULL	WIK	VAT	0.160000	KRA	VAT_STD_16	VAT 16%

How to read: Rows 1 and 2 show the same logical rule (WIK_INTERNET_EXCISE) versioned across time — the older 10% rate was active until 2023-07-01, after which the 15% rate took over (NULL effective_until = currently active). A charge dated 2023-06-15 picks row 2 (10% excise); a charge dated 2024-01-15 picks row 1 (15% excise). Rows 3, 4, 5, 6 are Wananchi KE's other current tax rules: Internet VAT (16%), TV VAT (16% — note no excise on TV), Voice excise (10%) and Voice VAT (16%). UG, TZ, SN rate rows are not shown — each operator has its own rules with its own prefix. Rates for UG/TZ/SN are deployment-time configuration; verified regulator-specific rate values to be confirmed at operator onboarding.

tax_group

Column	Type	Notes
code	TEXT PK	Globally unique. Example: WIK_INTERNET, WIK_TV, WIK_VOICE, WUG_GENERAL, YASSN_FCS_GENERAL
operator_code	TEXT NOT NULL	Enum. Values: WIK / WUG / WTZ / YASSN

Column	Type	Notes
description	TEXT NULL	Internal label. Example: Wananchi Kenya — Internet services (excise + VAT cascade)
rounding_mode	TEXT NOT NULL DEFAULT HALF_EVEN	Java RoundingMode enum applied to each tax line. Values: HALF_EVEN (banker's, default) / HALF_UP (commercial) / HALF_DOWN / UP / DOWN
rounding_scale	INTEGER NOT NULL	Decimal places. Example: 2 for KES/TZS/XOF; 0 for UGX (no sub-units)
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	Row creation time. Example: 2026-05-18 10:30:00
created_by	TEXT NOT NULL	Admin user ID. Example: admin_user_42

Sample data:

code	operator	description	rounding_mode	rounding_scale
WIK_INTERNET	WIK	Wananchi KE — Internet services (excise + VAT cascade)	HALF_EVEN	2
WIK_TV	WIK	Wananchi KE — TV services (VAT only)	HALF_EVEN	2
WIK_VOICE	WIK	Wananchi KE — Voice services (excise + VAT cascade)	HALF_EVEN	2
WUG_GENERAL	WUG	Wananchi UG — general VAT	HALF_EVEN	0

How to read: Wananchi KE has three tax groups corresponding to the three triple-play product categories. Each uses banker's rounding and 2-decimal precision (KES). Wananchi UG uses a single general group with 0-decimal precision because UGX has no sub-units; the rounded tax amount is always an integer. Tanzania (WTZ) and Senegal (YASSN) groups are not shown here — they exist in their respective operator deployment databases.

tax_group_member

Column	Type	Notes
id	TEXT PK	ULID
tax_group_code	TEXT NOT NULL	FK to tax_group.code
tax_rule_code	TEXT NOT NULL	FK to tax_rule.code (logical code; the specific version is picked at compute time by date)
order_within_group	INTEGER NOT NULL	Order of application within the group; integer; lower = applied first. Example: 1 for excise, 2 for VAT (so VAT cascades over excise via BASE_PLUS_PRIOR)
base_method	TEXT NOT NULL	Enum. Values: BASE (apply to base only) / BASE_PLUS_PRIOR (apply to base + sum of all prior tax amounts in this group) / BASE_PLUS_EXCISE (apply to base + sum of prior tax amounts where tax_type=EXCISE only)
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	Row creation time. Example: 2026-05-18 10:30:00

Constraint: UNIQUE (tax_group_code, tax_rule_code) — a rule appears at most once in a group. UNIQUE (tax_group_code, order_within_group) — no two rules at the same order within one group.

Sample data:

id	tax_group	tax_rule	order	base_method
tgm_01HZ...1	WIK_INTERNET	WIK_INTERNET_EXCISE	1	BASE
tgm_01HZ...2	WIK_INTERNET	WIK_INTERNET_VAT	2	BASE_PLUS_PRIOR
tgm_01HZ...3	WIK_TV	WIK_TV_VAT	1	BASE
tgm_01HZ...4	WIK_VOICE	WIK_VOICE_EXCISE	1	BASE
tgm_01HZ...5	WIK_VOICE	WIK_VOICE_VAT	2	BASE_PLUS_PRIOR
tgm_01HZ...6	WUG_GENERAL	WUG_GENERAL_VAT	1	BASE

How to read: For Wananchi KE Internet, two members in cascade order — excise applies first (base method BASE, computed on the package base), then VAT applies second (base method BASE_PLUS_PRIOR, computed on base + excise). This is what produces the KE Internet cascade: 10000 base → 1500 excise → 1840 VAT (on 11500) → 3340 total tax. TV has only one member (VAT only, no excise). Voice mirrors Internet structurally but with different rates. Wananchi UG has a single VAT rule applied to base.

tax_applicability_rule

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	Enum. Values: WIK / WUG / WTZ / YASSN
priority	INTEGER NOT NULL	Lower = higher priority. Example: 10 (highest), 100 (default), 1000 (fallback)
tax_group_code	TEXT NULL	The group to resolve, or NULL with is_exempt = true for an explicit exemption rule. Example: WIK_INTERNET
is_exempt	BOOLEAN NOT NULL DEFAULT FALSE	If true, this rule produces an EXEMPT_BY_RULE resolution (no taxes apply). tax_group_code MUST be NULL when true
drl_condition	TEXT NOT NULL	The DRL when clause body for this rule. Example: \$ctx : TaxContext(taxableKind == "SERVICE", taxableRef matches "INTERNET_.*") — the resolver wraps this in a full DRL rule at KJAR build time
display_label	TEXT NULL	Human-readable description. Example: Internet services → WIK_INTERNET tax group
effective_from	TIMESTAMP NOT NULL	Same effective-date semantics as tax rules. Example: 2023-07-01 00:00:00
effective_until	TIMESTAMP NULL	NULL = open-ended
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	Row creation time. Example: 2026-05-18 10:30:00
created_by	TEXT NOT NULL	Admin user ID

Indexes: PK; (operator_code, priority, effective_from DESC); UNIQUE (operator_code, priority)
WHERE effective_until IS NULL (no ties on priority for active rules within an operator).

Sample data:

id	operator	priority	tax_group	is_exempt	drl_condition	display_label
tar_01HZ...1	WIK	10	NULL	TRUE	\$ctx : TaxContext(customerAttributes ["tax_exempt"] == "true")	Tax-exempt customers (any service)
tar_01HZ...2	WIK	100	WIK_INTERNET	FALSE	\$ctx : TaxContext(taxableKind == "PACKAGE", taxableRef matches "PKG_INTERNET_.*")	Internet packages → WIK_INTERNET group
tar_01HZ...3	WIK	110	WIK_TV	FALSE	\$ctx : TaxContext(taxableKind == "PACKAGE", taxableRef matches "PKG_TV_.*")	TV packages → WIK_TV group
tar_01HZ...4	WIK	120	WIK_VOICE	FALSE	\$ctx : TaxContext(taxableKind == "PACKAGE", taxableRef matches "PKG_VOICE_.*")	Voice packages → WIK_VOICE group

id	operator	priority	tax_group	is_exempt	drl_condition	display_label
tar_01HZ...5	WIK	1000	WIK_INTERNET	FALSE	\$ctx : TaxContext(taxableKind == "FEE")	Fallback — fees taxed as internet

How to read: For Wananchi KE, the applicability ruleset is evaluated by priority. Priority 10 is a customer-level exemption (e.g., a tax-exempt government account) — matches BEFORE any category rule, so an exempt customer skips taxation entirely regardless of what they're buying. Priorities 100/110/120 dispatch by package prefix (PKG_INTERNET_* → WIK_INTERNET, etc.). Priority 1000 is a fallback that catches generic fees and applies the Internet group as default — pragmatic operator choice since most fees are network-related; documented for the operator to decide if it suits their accounting. A charge for PKG_TV_BASIC_HD on a non-exempt residential customer evaluates rules in order: 10 (no match — not exempt), 100 (no match — not Internet), 110 (match — TV package); resolves to WIK_TV group. Done.

API surface

Compute tax

```
POST /api/tax/compute HTTP/1.1
Authorization: Bearer <internal-svc-JWT>
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "taxableKind": "PACKAGE",
  "taxableRef": "PKG_INTERNET_FIBER_20M",
  "baseAmount": 10000.00,
  "currency": "KES",
  "customerCategory": "RESIDENTIAL",
  "customerAttributes": {
    "tax_exempt": "false"
  },
  "taxableAt": "2026-05-18T10:30:00Z",
  "forceTaxFree": false
}
```

Response 200 (cascade applied):

```
{
  "resolutionStatus": "RESOLVED",
  "resolvedTaxGroup": "WIK_INTERNET",
  "baseAmount": 10000.00,
  "taxLines": [
    {
      "code": "WIK_INTERNET_EXCISE",
      "displayName": "Excise Duty (Internet)",
      "taxType": "EXCISE",
      "rate": 0.150000,
      "baseAmount": 10000.00,
      "taxAmount": 1500.00
    },
    {
      "code": "WIK_INTERNET_VAT",
      "displayName": "VAT 16%",
      "taxType": "VAT",
      "rate": 0.160000,
      "baseAmount": 11500.00,
      "taxAmount": 1840.00
    }
  ],
  "totalTaxAmount": 3340.00,
  "totalWithTax": 13340.00,
  "currency": "KES"
}
```

Response 200 (exemption — customer attribute):

```
POST /api/tax/compute
{
  "operatorCode": "WIK",
  "taxableKind": "PACKAGE",
  "taxableRef": "PKG_INTERNET_FIBER_20M",
  "baseAmount": 10000.00,
  "currency": "KES",
  "customerCategory": "GOVERNMENT",
  "customerAttributes": { "tax_exempt": "true" },
  "taxableAt": "2026-05-18T10:30:00Z"
}
```

```
{
  "resolutionStatus": "EXEMPT_BY_CUSTOMER_ATTRIBUTE",
  "resolvedTaxGroup": null,
  "baseAmount": 10000.00,
  "taxLines": [],
  "totalTaxAmount": 0.00,
  "totalWithTax": 10000.00,
  "currency": "KES"
}
```

Response 200 (VAT-only — TV):

```
{
  "resolutionStatus": "RESOLVED",
  "resolvedTaxGroup": "WIK_TV",
  "baseAmount": 10000.00,
  "taxLines": [
    {
      "code": "WIK_TV_VAT",
      "displayName": "VAT 16%",
      "taxType": "VAT",
      "rate": 0.160000,
      "baseAmount": 10000.00,
      "taxAmount": 1600.00
    }
  ],
  "totalTaxAmount": 1600.00,
  "totalWithTax": 11600.00,
  "currency": "KES"
}
```

Response 400 (validation error):

```
{
  "error": "NEGATIVE_BASE_NOT_SUPPORTED",
  "detail": "baseAmount must be non-negative; received -5000.00. Use the absolute value and BIL-02-ADJ-01 handles credit r
}
```

Admin: list rules for operator

GET /api/tax/admin/rules?operatorCode=WIK&activeOn=2026-05-18

```
{
  "rules": [
    {
      "code": "WIK_INTERNET_EXCISE",
      "effectiveFrom": "2023-07-01T00:00:00Z",
      "effectiveUntil": null,
      "operatorCode": "WIK",
      "taxType": "EXCISE",
      "rate": 0.150000,
      "regulator": "KRA",
      "regulatorTaxCode": "EXC_INT_15",
      "displayName": "Excise Duty (Internet)"
    }
  ]
}
```

Admin: add new rule version (rate change)

POST /api/tax/admin/rules HTTP/1.1
 Authorization: Bearer <admin-JWT>
 Content-Type: application/json

```
{
  "code": "WIK_INTERNET_EXCISE",
  "effectiveFrom": "2027-01-01T00:00:00Z",
  "effectiveUntil": null,
  "operatorCode": "WIK",
  "taxType": "EXCISE",
  "rate": 0.200000,
  "regulator": "KRA",
  "regulatorTaxCode": "EXC_INT_20",
  "displayName": "Excise Duty (Internet)",
  "description": "Per Finance Act 2026, Sec X – rate raised from 15% to 20% effective 2027-01-01"
}
```

The admin endpoint atomically: (a) sets the previous active version's `effective_until` to the new rule's `effective_from`; (b) inserts the new row; (c) emits `sophix.config.tax.changed`. If the existing version's `effective_until` was already non-null and after `effective_from`, the request is rejected with `EFFECTIVE_DATE_OVERLAP`.

Admin: tax group CRUD

POST /api/tax/admin/groups, GET /api/tax/admin/groups, PUT /api/tax/admin/groups/{code}, POST /api/tax/admin/groups/{code}/members, DELETE /api/tax/admin/groups/{code}/members/{rule_code}.

Admin: applicability rule CRUD

POST /api/tax/admin/applicability-rules — accepts the DRL condition as a string. The catalog service validates the DRL syntactically (KIE compiler check) before storing. On success, an updated KJAR is built and pushed to the KIE Server.

Kafka events

sophix.config.tax.changed

```
{
  "eventType": "TaxConfigChanged",
  "aggregateId": "WIK",
  "timestamp": "2026-05-18T10:30:00Z",
  "payload": {
    "operatorCode": "WIK",
    "changeType": "RULE_ADDED",
    "affectedCodes": ["WIK_INTERNET_EXCISE"],
    "actor": "admin_user_42",
    "changeSummary": "Added new effective version of WIK_INTERNET_EXCISE at 0.200000 from 2027-01-01"
  }
}
```

Consumers (BIL-01 Core, BIL-02-GEN-01, BIL-02-ADJ-01) invalidate their tax-config caches on receipt.

Cross-DD coordination

Touched	What PLM-CFG-02 owns	What the other DD owns
BIL-01 Core	Provides POST /api/tax/compute called per R-BIL-01-AM-5 on every charge with non-zero gross. Returns tax lines that BIL-01 records in charge_ledger.tax_amount_total and outcome_envelope.	Owns the charge pipeline and charge_ledger.
BIL-02-GEN-01	Provides POST /api/tax/compute for invoice-line-level tax recomputation (rare — only when an invoice is reissued with corrected base amounts).	Owns invoice generation and tax line storage on invoices.
BIL-02-TAX-01	Provides regulator_tax_code and regulator metadata on each tax rule. TAX-01 uses these when submitting tax invoice line items to eTIMS/EFD/etc.	Owns tax invoice generation, regulator submission, signing.
BIL-02-ADJ-01	Provides POST /api/tax/compute called with absolute value when issuing tax-bearing credit/debit notes; the returned tax becomes a tax credit/debit in the note.	Owns adjustment workflow.
PLM-CFG-01 Service Catalog	Reads via Drools applicability rules — the rules can match on taxableRef patterns derived from Service codes.	Owns Service catalog.
SIP-01 Package Management	Reads via applicability — Package codes (PKG_INTERNET_* etc.) drive rule matching.	Owns Package catalog.
Drools KIE Server	Hosts the tax-applicability KJAR.	Foundation.

Dependencies

Dependency	Owner	Status
BIL-01 Core	billing team	✓ v0.9 (caller)
BIL-02-GEN-01	billing team	✓ v1.3
BIL-02-TAX-01	billing team	✓ v1.4
BIL-02-ADJ-01	billing team	✓ v1.3
PLM-CFG-01 Service Catalog	plm team	✓ v0.8
SIP-01 Package Management	plm team	✓ v1.0
FOUNDATION_DROOLS	infra team	✓

Dependency	Owner	Status
FOUNDATION_KAFKA	infra team	✓
FOUNDATION_CACHE	infra team	✓
FOUNDATION_AUTH	infra team	✓

B — Current TZ

Workshop materials do not describe a centralized tax configuration service in TZ Sophix. Tax computation appears to live in scattered hardcoded methods across billing modules, with rates baked into code constants or per-package metadata. Tax invoice generation (TZ EFD integration) is handled by a separate post-payment script. No verified TZ baseline for the structure of the tax configuration; this DD designs the V3 target without claiming a documented TZ parallel.

C — Gap & bridge

PLM-CFG-02 is new functionality. It introduces:

- Centralized tax rate catalog with operator-scoped, time-versioned rules
- Cascading tax computation via configurable base methods (BASE / BASE_PLUS_PRIOR / BASE_PLUS_EXCISE)
- Drools-based applicability resolution decoupling tax rules from billing code
- Stateless compute service consumable by BIL-01, BIL-02-ADJ-01, BIL-02-GEN-01
- Explicit regulator metadata (regulator + regulator_tax_code) enabling BIL-02-TAX-01's eTIMS/EFD/etc. submission
- Customer-level and admin-level exemption mechanisms
- Config change events driving cache invalidation across consumers
- Effective-date versioning so rate changes (e.g., KE Internet excise 10% → 15% in 2023) are accurately applied to historical and future charges

Cleanup work the catalog needs at operator onboarding:

- Wananchi KE: confirmed rate set (Internet excise 15% + VAT 16%; TV VAT 16%; Voice excise 10% + VAT 16%)
- Wananchi UG / TZ: rates to be confirmed at deployment time against URA / TRA published rate sheets
- Yas Senegal: rates to be confirmed at deployment time against DGID published rate sheets
- Per-operator KJAR ({OPERATOR}_applicability.drl) seeded with package-prefix dispatch rules at go-live